"Never trust the client -- verify everything on the server."

MODULE 36

Secure Frontend Communication

Protect the CRM single-page application from token theft, XSS, and CSRF -- and enforce every authorization decision on the server, not the browser.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 36 OVERVIEW

Harden the CRM single-page application against token theft, XSS, and CSRF -- while keeping the Spring API as the only source of truth for authorization.

The browser is untrusted territory. This module covers frontend threats and defenses, then a hands-on lab that hardens the real CRM SPA's login, token storage, and API calls.

DURATION & LAB

1 Hour Theory

Frontend threats, OWASP risks, JWT review, token storage, XSS/CSRF, secure API calls, and best practices.

1.5 Hours Lab

Frontend Security for the CRM SPA: threat model, in-memory tokens, origin-scoped auth, XSS proof, CSRF/CSP evidence.

Scenario

The CRM SPA holds Amina Khan's and Ravi Singh's PII in the browser -- a high-value target for token theft and XSS.

MODULE ARC

Understand the Threats

The frontend attack surface, OWASP frontend risks, and why the browser is never trusted.

Master Token Handling

JWT structure, secure storage, expiration and refresh, and secure API call patterns.

Build the Lab

Harden the real CRM SPA: memory tokens, origin-scoped bearer, XSS proof, CSRF/CSP evidence.

KEY TAKEAWAY Total time: 2.5 hours (1 hour theory + 1.5 hours hands-on lab). By the end, you will secure a React SPA's authentication, token handling, and API communication.

WHY FRONTEND SECURITY MATTERS

The frontend is the first point of interaction between users and your application. If it's not secure, attackers can steal data, hijack sessions, and misuse your application -- no matter how strong your backend is.

Security is a shared responsibility: a secure backend with an insecure frontend still puts users, data, and business reputation at risk.

INSECURE FRONTEND vs SECURE FRONTEND

Insecure Frontend

XSS, CSRF, data theft, session hijacking. Leads to data breaches, financial loss, legal issues, and loss of user trust.

Secure Frontend

Validated input, secure sessions, secure cookies, safe API calls. Builds trust, ensures compliance, delivers a secure experience.

WHAT'S AT STAKE

- **1. Protect User Data —** attackers can steal sensitive information like tokens, PII, and payment details.
- **2. Prevent Account Takeover —** weak frontend security can let attackers hijack sessions and gain unauthorized access.
- **3. Avoid Financial & Legal Impact —** breaches lead to financial loss, regulatory penalties, and costly incident response.
- **4. Maintain User Trust —** security incidents damage your reputation and erode user confidence.
- **5. Ensure Compliance —** strong frontend security helps meet industry standards and regulatory requirements.

KEY TAKEAWAY A secure frontend is essential for protecting users, data, and business value. By addressing frontend risks proactively, you build secure, trustworthy, and resilient web applications.

Why Frontend Security Matters

The frontend is the first line of interaction—and the first line of defense.

Key Reasons

Protects Users
Safeguards users' data, privacy, and identity.

Prevents Data Breaches
Stops attackers from stealing sensitive information.

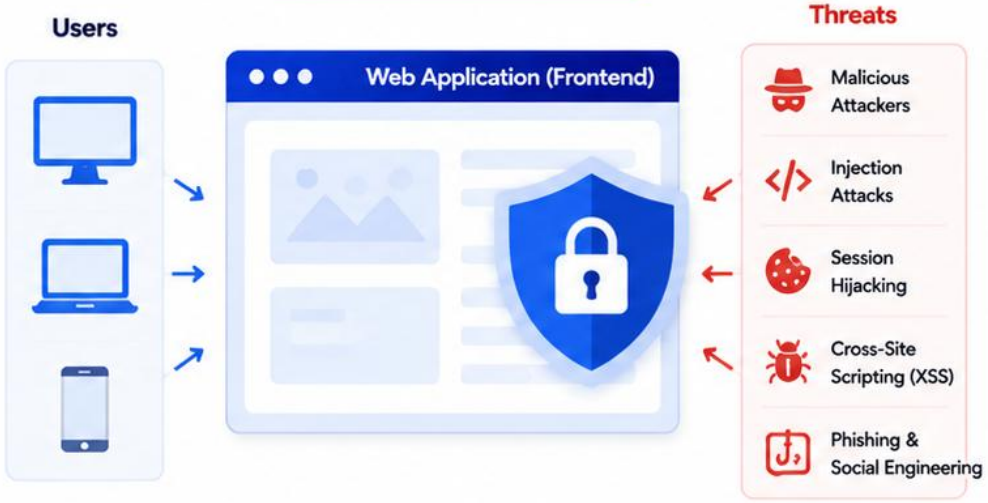
Reduces Attack Surface
Mitigates risks from client-side vulnerabilities.

Builds Trust
Users trust secure applications and brands.

Ensures Compliance
Helps meet security standards and legal requirements.

Prevents Business Impact
Avoids financial loss, downtime, and reputational damage.

The Frontend is Exposed



What's at Risk?

User Data · Credentials · Business Data · Reputation

Consequences of Weak Frontend Security

Data Theft
Attackers can steal sensitive user and business data.

Account Takeover
Compromised sessions and tokens lead to account takeovers.

Financial Loss
Fraud, fines, and recovery costs can be significant.

Reputational Damage
Loss of customer trust and brand credibility.

Legal & Compliance Risk
Violations can lead to legal action and penalties.

Frontend Security Best Practices

- Validate all user inputs on client and server
- Sanitize output to prevent XSS
- Use HTTPS everywhere
- Secure tokens and manage sessions safely
- Implement Content Security Policy (CSP)
- Keep dependencies updated
- Follow the principle of least privilege

MODULE LEARNING OBJECTIVES

By the end of this module, you will understand frontend security risks and apply best practices to build secure, resilient, user-centric web applications.

AFTER COMPLETING THIS MODULE, YOU WILL BE ABLE TO:

1. Understand Threats

Identify common frontend security threats and OWASP frontend risks that impact web apps.

2. Secure Auth with JWT

Review JWT authentication and apply best practices for secure token usage.

3. Secure Token Storage

Store tokens securely in the browser and implement expiration/refresh strategies.

4. Prevent XSS Attacks

Understand Cross-Site Scripting and apply techniques to prevent and mitigate it.

5. Prevent CSRF Attacks

Understand Cross-Site Request Forgery and implement effective protection mechanisms.

6. Design Secure UIs

Apply secure UI design principles to minimize exposure and reduce attack surface.

7. Protect Sensitive Data

Identify sensitive data in the frontend and use techniques to protect it from access.

8. Secure API Calls

Secure API requests and responses using HTTPS, proper headers, and safe data handling.

9. Auth Flows

Build secure login, logout, and token refresh flows with proper session management.

10. Enterprise Practices

Apply frontend security best practices and a checklist for secure, compliant apps.

KEY TAKEAWAY Goal: build secure, trustworthy, and resilient web applications by understanding frontend security risks and implementing industry-standard best practices.

FRONTEND THREAT LANDSCAPE (1 OF 2)

Modern web applications run in the browser, which is inherently less trusted. Attackers can target the frontend to steal data, hijack sessions, and manipulate your application.

WHO ARE THE ATTACKERS?

- **External Attackers —** malicious individuals scanning the internet for vulnerable web applications.
- **Malicious Insiders —** authorized users who abuse access or extract sensitive information.
- **Automated Bots —** tools and scripts that discover vulnerabilities and launch attacks at scale.
- **Social Engineers —** trick users into clicking malicious links or sharing credentials.

FRONTEND ATTACK SURFACES

- **Client-Side Code —** vulnerabilities in JavaScript, libraries, and frameworks.
- **Browser Storage —** localStorage, sessionStorage, cookies, and IndexedDB.
- **Cookies & Tokens —** theft or misuse of authentication artifacts.
- **Network Requests —** API calls can be intercepted, modified, or replayed.
- **User Inputs —** forms, URL params, file uploads, and rich text.
- **Third-Party Scripts —** external scripts can introduce vulnerabilities.

KEY TAKEAWAY The frontend is a powerful platform -- but also an open attack surface. The browser is often the easiest entry point.

Frontend Threat Landscape

Modern web applications face diverse threats targeting the client-side.

Common Frontend Threats

Cross-Site Scripting (XSS)
Attackers inject malicious scripts that run in users' browsers.

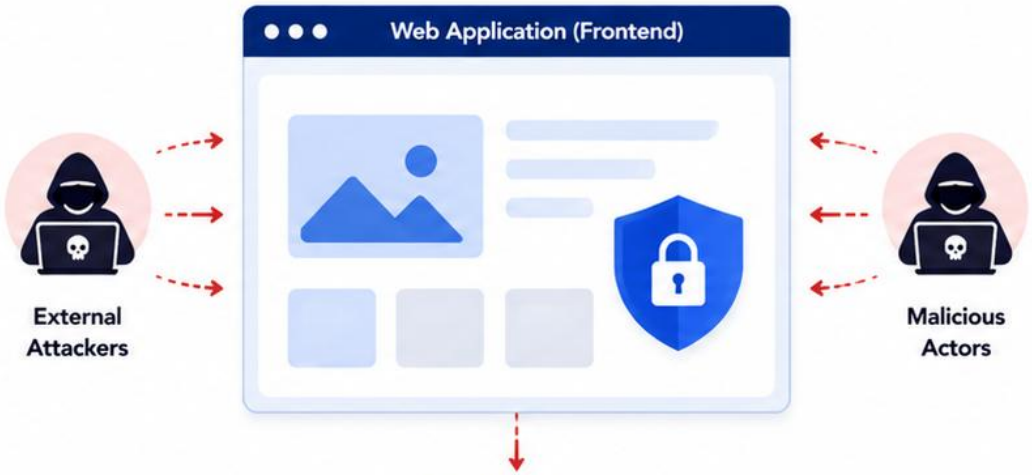
Cross-Site Request Forgery (CSRF)
Attackers trick users into performing unwanted actions.

Session Hijacking
Attackers steal session cookies or tokens to impersonate users.

Injection Attacks
Malicious input is inserted into client-side code or APIs.

Insecure Data Storage
Sensitive data stored in insecure localStorage, cookies, or IndexedDB.

Third-Party Risks
Vulnerable or malicious third-party scripts and ilbraries.



Business Impact

Financial Loss
Fraud, chargebacks, and unauthorized transactions.

Data Breach
Exposure of sensitive user and business data.

Reputation Damage
Loss of customer trust and brand credibility.

Legal & Compliance Risk
Regulatory penalties and legal consequences.

Operational Disruption
Service downtime and business interruption.

Key Takeaway
Secure the frontend to protect users, data, and your business.

Frontend Security Best Practices

- Validate and sanitize all user inputs
- Use secure cookies (HttpOnly, SameSite, Secure)
- Implement Content Security Policy (CSP)
- Avoid storing sensitive data in localStorage
- Keep dependencies and libraries updated
- Implement strong authentication and authorization

FRONTEND THREAT LANDSCAPE (2 OF 2)

What's at risk when the frontend attack surface is exploited, and the best practices that close it.

WHAT'S AT RISK?

Risk	Impact
Data Breach	Exposure of sensitive user data and business information.
Account Takeover	Attackers gain control of user accounts and sessions.
Financial Loss	Fraud, chargebacks, and revenue impact.
Legal & Compliance Risk	Violation of regulations like GDPR and PCI-DSS.
Reputation Damage	Loss of user trust and brand credibility.

BEST PRACTICES

- Identify and assess frontend risks early.
- Secure code, dependencies, and configurations.
- Validate all inputs and handle errors securely.
- Secure API communication (HTTPS, CORS).
- Monitor, test, and keep your application updated.

LAB 36 CONNECTION

Lab 36's threat model document maps each of these risks to a concrete control for the CRM SPA -- assets, attacker goals, and mapped controls, before any code is written.

KEY TAKEAWAY Security must be built into the frontend, not just the backend. Even small vulnerabilities can lead to major breaches.

TRY IT YOURSELF — EXERCISE 1: THREAT SKETCH

Reinforces: mapping frontend threats and the UI-vs-API authorization boundary for the Northstar CRM SPA -- an analysis exercise.

ASSETS ATTACKERS WANT

- **Session Tokens —** the in-memory access token issued after login -- if stolen, an attacker can impersonate the user.
- **Customer PII —** CUS-1001 Amina Khan (ACTIVE) and CUS-1002 Ravi Singh (PROSPECT), rendered directly in the browser.
- **Admin Actions —** any privileged route or button an attacker could reach by manipulating the client.

STEPS (IN notes/lab36-security.md)

Step	What To Do
1 — Assets	List what attackers want: session tokens, customer PII for Amina/Ravi, and admin actions.
2 — Threats	Name at least three threats: XSS, token theft, CSRF (if a cookie session is used), and open redirects.
3 — UI vs API	Write one sentence: hiding a button is not authorization -- Spring must enforce it.
4 — Save	Save your threat list and the UI-vs-API sentence to notes/lab36-security.md.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-threat-sketch.md (workspace: examples/module-36-exercises). Pass criteria: at least 3 threats named, the authorization boundary stated, and notes saved.

OWASP FRONTEND RISKS (1 OF 2)

The OWASP Top 10 for Web provides a baseline for securing web applications. These are the risks most relevant to frontend applications and client-side code.

OWASP (Open Worldwide Application Security Project) is a global non-profit foundation that works to improve the security of software.

OWASP Risk	Description	Frontend Impact	Example Scenario
A03: Injection (e.g., XSS)	Untrusted data is sent to an interpreter as part of a command or query.	Attackers inject malicious scripts (XSS) and steal data, tokens, or perform actions.	User input reflected in the page without sanitization, leading to script execution.
A05: Security Misconfiguration	Missing or insecure configuration in frameworks, libraries, or the application.	Exposed debug info, insecure headers, or default settings can be exploited.	Debug mode enabled in production exposes stack traces and internal details.
A07: Identification & Auth Failures	Weak authentication, session management, or token handling issues.	Attackers can hijack sessions, bypass authentication, or gain unauthorized access.	Tokens stored in localStorage are stolen via XSS and used to impersonate the user.

KEY TAKEAWAY Attackers target the frontend because it's the easiest entry point. Secure code, secure configurations, and secure dependencies are your first line of defense.

OWASP Frontend Risks

Top risks for frontend applications based on OWASP Top 10 for Web (2021) with frontend focus.

1 Cross-Site Scripting (XSS)

What it is: Attackers inject malicious scripts into web pages viewed by other users.

Example: Storing user input and rendering it without sanitization.

2 Injection

What it is: Untrusted data is sent to interpreters as part of a command or query.

Example: Passing unsanitized input to APIs, SQL, NoSQL, or OS commands.

3 Broken Access Control

What it is: Restrictions on what users can do are not properly enforced.

Example: Exposing admin features in the UI or trusting client-side role checks.

4 Insecure Design

What it is: Security is not considered early in the design of frontend features.

Example: Missing threat modeling for new UI flows or components.

5 Security Misconfiguration

What it is: Frontend or hosting is misconfigured, exposing sensitive information.

Example: Enabling directory listing, exposing source maps, or weak CORS settings.

6 Vulnerable & Outdated Components

What it is: Using libraries or frameworks with known vulnerabilities.

Example: Outdated npm packages with published security flaws.

7 Identification & Authentication Failures

What it is: Weak or missing authentication mechanisms in the frontend.

Example: Storing tokens insecurely or poor session management.

8 Software & Data Integrity Failures

What it is: Code or data is modified without proper verification.

Example: Loading third-party scripts without integrity checks (Subresource Integrity missing).

9 Security Logging & Monitoring Failures

What it is: Insufficient logging and monitoring of frontend security events.

Example: Not capturing client-side errors or suspicious activities.

10 Server-Side Request Forgery (SSRF)

What it is: The server is tricked into making requests to unintended systems via the frontend.

Example: Proxy features that allow users to fetch arbitrary URLs.

Why These Matter for Frontend

Frontend is the first point of interaction and attack.

Client-side flaws can lead to account takeover, data theft, and fraud.

Modern SPAs and JS-heavy apps increase the attack surface.

Secure frontend + secure backend = defense in depth.

Helps meet compliance and protects brand reputation.

Frontend Security Best Practices

- Validate and sanitize all user inputs
- Store tokens securely (HttpOnly, Secure, SameSite)
- Use Content Security Policy (CSP)
- Keep dependencies updated
- Enforce strict CORS and security headers
- Monitor and log frontend errors and anomalies

OWASP FRONTEND RISKS (2 OF 2)

The remaining two OWASP risk categories most relevant to the frontend, and the prevention measures common to all five.

OWASP Risk	Description	Frontend Impact	Example Scenario
A08: Software & Data Integrity Failures	Using vulnerable libraries or loading untrusted scripts compromises integrity.	Malicious libraries or scripts can modify frontend behavior or steal data.	A compromised third-party JS library is loaded and executes malicious code.
A01: Broken Access Control	Improper restrictions allow users to access unauthorized resources or actions.	Manipulating client-side logic can expose hidden features or data.	Users modify API requests in DevTools to try to access admin features.

KEY PREVENTION ACROSS ALL FIVE RISKS

- **Validate & Sanitize Input**
- **Use Trusted, Pinned Dependencies**
- **Enforce Access Control on the Backend**
- **Apply Content Security Policy**
- **Keep Frameworks & Libraries Updated**

LAB 36 CONNECTION

A01 Broken Access Control is the exact risk ProtectedRoute alone cannot fix: a user can edit DevTools or the URL, but the Spring API must still reject the request. This is why Lab 36's threat model states, explicitly, that route guards are not authorization.

KEY TAKEAWAY Frontend security is not just about UI -- it's about protecting users, data, and business logic from real-world threats.

JWT AUTHENTICATION REVIEW (1 OF 2)

JSON Web Token (JWT) is a compact, self-contained way to securely transmit information between parties -- widely used for authentication and authorization in modern web apps.

HOW JWT WORKS

1. **Login** — Submits credentials.
2. **Authenticate** — Backend validates.
3. **Generate JWT** — Signed token created.
4. **Store Token** — Client stores it.
5. **Send Requests** — Authorization header.
6. **Validate** — Backend verifies signature.

JWT STRUCTURE -- THREE PARTS

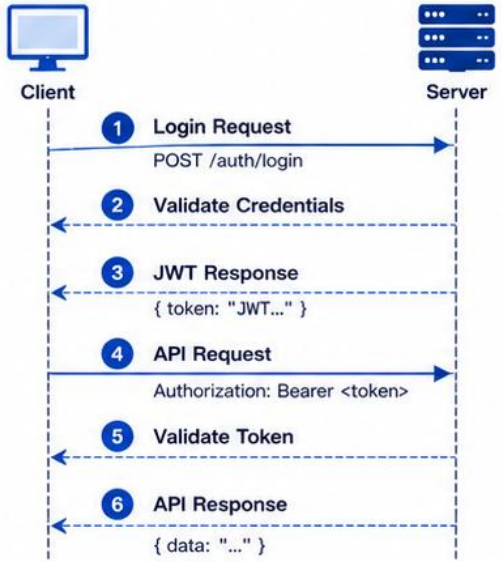
Part	Contains	Example
1. Header	Token type and signing algorithm.	{"alg":"HS256","typ":"JWT"}
2. Payload (Claims)	Claims / information about the user and token.	{"sub":"123","role":"USER","iat":....}
3. Signature	Ensures the token has not been tampered with.	HMACSHA256(header + "." + payload, secret)

KEY TAKEAWAY The backend signs the token using a secret key. Any change to the token will make the signature invalid. JWT provides authentication, not authorization -- always enforce permissions on the backend.

JWT Authentication Review

JSON Web Token (JWT) based authentication flow and key concepts

1. Authentication Flow



2. JWT Structure

A JWT consists of three parts separated by dots (.)

Header . Payload . Signature

Base64Url | Base64Url | Base64Url

Header
- Contains token type and signing algorithm
- Example: `{ "alg": "HS256", "typ": "JWT" }`

Payload
- Contains claims (user data and metadata)
- Example: `{ "sub": "123", "name": "John", "role": "user", "exp": 1710000000 }`

Signature
- Ensures token integrity and authenticity
- Created using: HMACSHA256(base64Url(header.payload), secret)

3. How JWT Works

User Login
User submits credentials to login.

Server Validates
Server validates credentials.

JWT Generated
Server generates a JWT and sends it to the client.

Client Stores Token
Client stores the JWT (e.g., in localStorage or httpOnly cookie).

API Requests
Client includes the JWT in the Authorization header for API calls.

Server Validates Token
Server verifies the JWT. If valid, processes the request.

4. Advantages

Stateless
No server-side session storage required.

Scalable
Easy to scale across multiple servers.

Cross-Domain
Works well across different domains and services (CORS-friendly).

5. Best Practices

- Use HTTPS always
- Set short expiration time (exp)
- Use strong secrets and algorithms (HS256 / RS256)
- Store tokens securely (httpOnly cookies preferred)
- Implement token refresh mechanism

6. Common Claims

Claim	Description	Example
iss	Issuer	"https://myapp.com"
sub	Subject (User ID)	"12345"
aud	Audience	"myapi"
exp	Expiration Time	1710000000
iat	Issued At	1709996400
nbf	Not Before	1709996400
jti	JWT ID	"abc-123-xyz"

7. Key Points

- JWT is a compact, URL-safe token format for secure data exchange.
- It is digitally signed to ensure integrity and authenticity.
- Always validate tokens on the server.
- Do not store sensitive data in the payload.

JWT AUTHENTICATION REVIEW (2 OF 2)

Why teams choose JWT, and the best practices that keep it safe on the frontend.

BENEFITS OF JWT

- **Stateless Authentication —** no need to store session information on the server.
- **Scalable —** works across multiple services and servers.
- **Secure —** digitally signed; cannot be modified without invalidating the signature.
- **Compact —** small in size and easily transmitted in HTTP headers.

BEST PRACTICES

- **Store tokens securely —** prefer HttpOnly cookies or in-memory storage over localStorage.
- **Use HTTPS —** always transmit tokens over secure channels.
- **Set expiration (exp) —** keep token lifetime short (e.g., 15-30 minutes).
- **Use refresh tokens —** issue a separate refresh token for long-lived sessions.
- **Validate on each request —** always verify signature and expiration.
- **Avoid sensitive data in payload —** do not store passwords or PII in the token.

KEY TAKEAWAY JWT provides authentication, not authorization. Always enforce permissions on the backend based on user roles and scopes.

SECURE TOKEN STORAGE (1 OF 2)

How you store JWTs on the client side directly impacts the security of your application. Choose the right storage mechanism and apply strong safeguards.

Key principle: store tokens in the safest location available in the browser, minimize exposure, and reduce the risk of theft via XSS or other attacks.

Storage Option	How It Works	XSS Risk	CSRF Risk	Recommendation
Cookies (HttpOnly)	Stored in cookies with HttpOnly flag; not accessible via JavaScript.	Low	High	Recommended for most scenarios (with CSRF protection).
localStorage	Stored in browser localStorage; accessible via JavaScript.	High	Low	Avoid storing sensitive tokens.
sessionStorage	Stored in sessionStorage; accessible via JavaScript.	High	Low	Avoid for sensitive tokens.
In-Memory (Variable)	Stored in JS memory (e.g., a variable or state).	Medium	Low	Acceptable for short-lived access tokens -- Lab 36's approach.
IndexedDB	Stored in the browser's IndexedDB database.	High	Low	Avoid for sensitive tokens.

KEY TAKEAWAY The safest place for JWTs on the client is an HttpOnly, Secure, SameSite cookie combined with CSRF protection. Always minimize token exposure and validate on the server.

Secure Token Storage

Store tokens securely to prevent theft, misuse, and unauthorized access.

HTTP-Only Cookies

Stored in cookies with HttpOnly and Secure flags.
Not accessible via JavaScript.

Secure Storage (Web)

Use Web Storage (localStorage) or Session Storage with caution.
Avoid storing long-lived tokens.

Secure Storage (Mobile)

Use platform-provided secure storage like Keychain (iOS) or Keystore (Android).

In-Memory Storage

Store tokens in memory only (e.g., React state, variables).
Lost on refresh but safer.

BEST PRACTICES

Use Short-Lived Tokens

Minimize token lifetime to reduce risk.

Implement Token Rotation

Rotate refresh tokens regularly.

Enforce HTTPS

Always transmit tokens over secure channels.

Validate Tokens on Server

Always verify token signature, issuer, and expiration.

Avoid localStorage for Sensitive Tokens

More vulnerable to XSS attacks.

Secure storage + best practices = Stronger protection against token theft and unauthorized access.

SECURE TOKEN STORAGE (2 OF 2)

Best practices for whichever storage mechanism you choose, and the concrete risks to avoid.

BEST PRACTICES

- **Use Secure & SameSite Flags —** always set Secure to true and SameSite=Lax or Strict for cookies.
- **Use Short-Lived Access Tokens —** keep access tokens short-lived and use refresh tokens to obtain new ones.
- **Rotate Refresh Tokens —** rotate refresh tokens after each use and invalidate old ones.
- **Clear Tokens on Logout —** remove tokens from storage and invalidate them on the server.
- **Validate and Verify on Server —** never rely on token data alone -- always verify signature and claims.

RISKS TO AVOID

- Never store tokens in localStorage or sessionStorage (exposed to XSS).
- Avoid storing tokens in URLs or query parameters.
- Don't expose tokens in client-side logs or error messages.
- Sanitize all user inputs to prevent XSS.

LAB 36 CONNECTION

tokenStore.ts exposes only get/set/clear over one in-memory variable -- never mirrored to localStorage, sessionStorage, or logs. Logout calls tokenStore.clear() and clears the customer cache.

KEY TAKEAWAY Never store tokens in localStorage or sessionStorage (exposed to XSS attacks). Sanitize all user inputs, and always verify tokens on the server.

TRY IT YOURSELF — EXERCISE 2: TOKEN STORAGE OPTIONS

Reinforces: comparing token storage options and choosing + justifying an approach for the CRM SPA -- an architecture exercise.

REFERENCE: WHERE CAN A TOKEN LIVE?

Option	Risk / Note
In-Memory Variable	Lost on refresh; safer from XSS persistence -- Lab 36's approach.
sessionStorage	Per-tab; readable by any injected script (XSS).
localStorage	Survives refresh; readable by any injected script (XSS) -- banned in Lab 36.
HttpOnly Cookie	Not JavaScript-readable; needs a CSRF strategy.

STEPS (IN notes/lab36-security.md)

Step	What To Do
1 — Study the Table	Copy the reference table above into your notes.
2 — Lab Choice	Pick one approach for Lab 36 and justify it in two sentences.
3 — Never Rules	Write the never-commit rules: never commit real tokens; never put DB passwords in Vite env files.
4 — Fixture	Use the fake token lab-token-001 in your notes only -- never a real secret.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-token-storage.md (workspace: examples/module-36-exercises). Pass criteria: a justified choice, a stated never-commit rule, and a fake-token-only example.

TOKEN EXPIRATION AND REFRESH (1 OF 2)

Short-lived access tokens reduce risk if stolen, while refresh tokens allow a seamless user experience by issuing new access tokens securely.

TYPES OF TOKENS

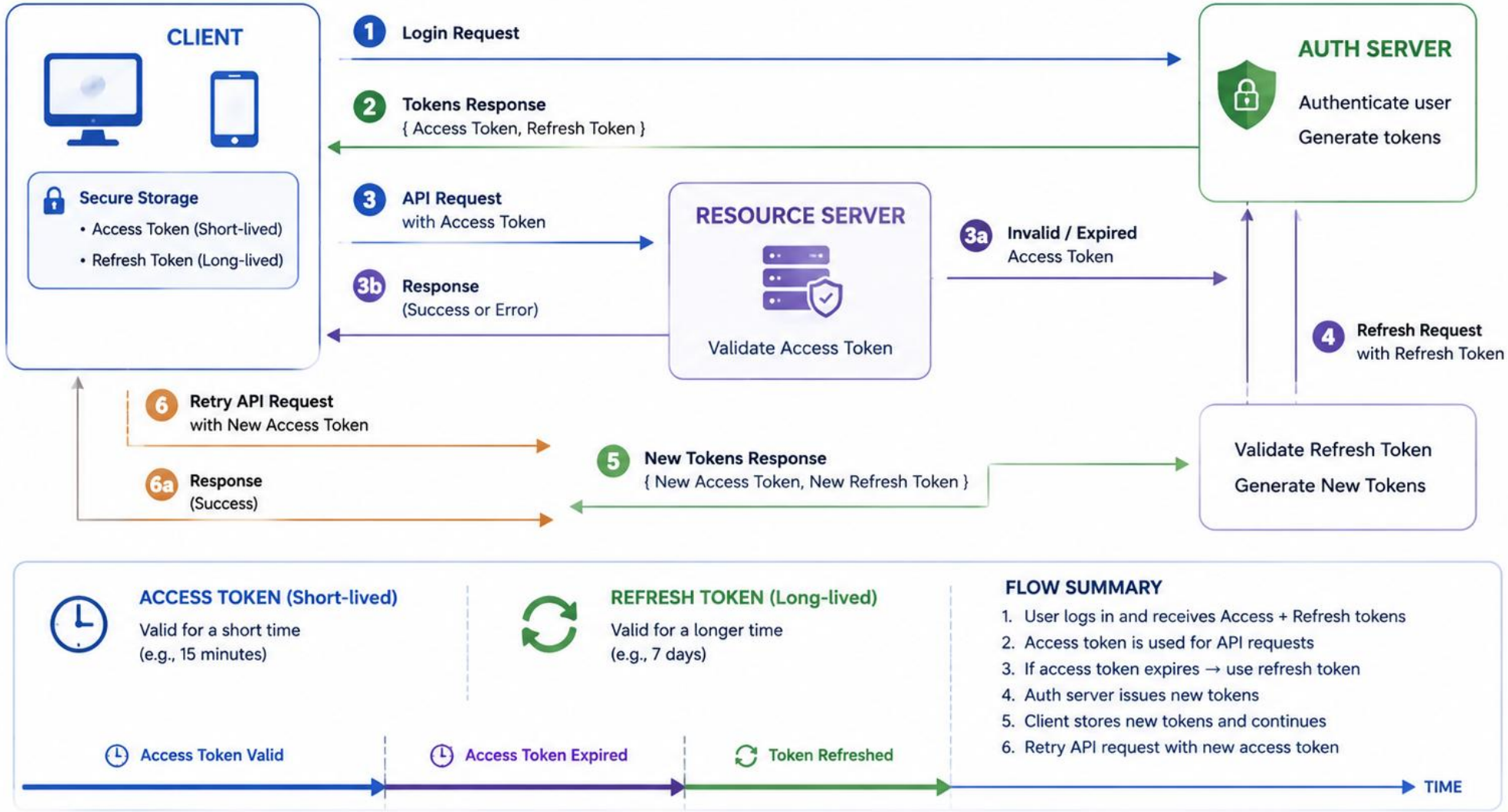
- **Access Token —** short-lived (e.g., 5-15 min). Used to access APIs and protected resources. Contains user claims and permissions.
- **Refresh Token —** long-lived (e.g., 7-30 days). Used only to get a new access token. Stored securely -- HttpOnly cookie recommended.

HOW EXPIRATION AND REFRESH WORKS

Key concept: access tokens are short-lived (minutes); refresh tokens are long-lived (days/weeks). This balances security with a seamless user experience.

KEY TAKEAWAY Use short-lived access tokens and secure refresh tokens to protect your application while ensuring smooth user experience. Always store, validate, and rotate securely.

Token Expiration and Refresh



TOKEN EXPIRATION AND REFRESH (2 OF 2)

Best practices, why refresh tokens matter, and common scenarios students will hit in Lab 36.

BEST PRACTICES

- **Use Short-Lived Access Tokens —** minimize risk by keeping access tokens short-lived.
- **Store Refresh Tokens Securely —** use HttpOnly, Secure, SameSite cookies.
- **Rotate Refresh Tokens —** issue a new refresh token whenever an old one is used.
- **Revoke on Logout —** invalidate refresh tokens when a user logs out or changes password.
- **Validate on Every Request —** always verify token signature, expiration, and claims.

WHY USE REFRESH TOKENS?

Enhanced Security **Better UX** **Balanced Approach**

Reduced Re-Auth

COMMON SCENARIOS (401 HANDLING)

- **Token expired while active —** use the refresh token to get a new access token.
- **Multiple tabs open —** an in-memory token is NOT shared across tabs -- a documented Lab 36 limitation.
- **Suspicious activity detected —** revoke the refresh token and force re-login.

KEY TAKEAWAY Use short-lived access tokens and secure refresh tokens. Always store, validate, and rotate securely -- and design for the retry and re-authentication paths, not only the happy path.

CROSS-SITE SCRIPTING (XSS) (1 OF 2)

XSS occurs when an attacker injects malicious scripts into web pages viewed by other users. These scripts run in the victim's browser and can steal cookies, tokens, or sensitive information.

HOW XSS WORKS

- 1. Attacker injects a malicious script into the application (via a form, comment, or search input).
- 2. The application stores or reflects the script without proper validation.
- 3. Another user visits the page, and the malicious script is delivered in their browser.
- 4. The script runs with the privileges of the user and can steal data, perform actions, or deface content.

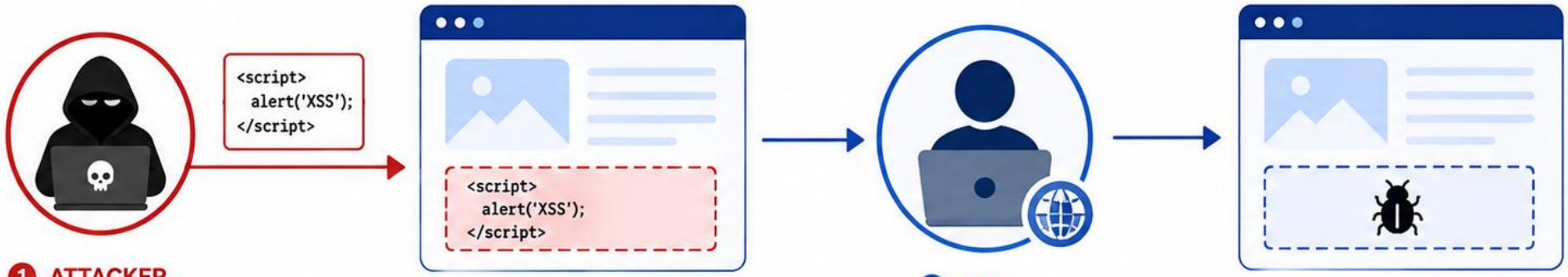
TYPES OF XSS

Type	How It Happens
Reflected XSS	Script is reflected off the server in the response, usually via a URL or form input.
Stored (Persistent) XSS	Script is stored on the server (e.g., in a DB field like fullName) and served to users later.
DOM-Based XSS	Client-side JS modifies the DOM based on user input, e.g. writing location.hash via innerHTML.

KEY TAKEAWAY Never trust user input. XSS exploits trust -- the attacker's script runs with the victim's own privileges inside their own browser.

Cross-Site Scripting (XSS)

An attacker injects malicious scripts into websites viewed by other users.



1 ATTACKER

Attacker crafts a malicious script and injects it into the target website.

2 VULNERABLE WEBSITE

The malicious script is stored (or reflected) on the server.

3 USER

A user visits the page. The malicious script is delivered in the page.

4 SCRIPT EXECUTION

The script runs in the user's browser with the privileges of the website.

HOW XSS WORKS

1. Attacker injects malicious script into a website (stored or reflected).
2. The website includes the script in a page without proper validation.
3. When another user visits the page, the script executes in their browser.
4. The attacker can steal data, perform actions, or manipulate the page.

- Steal Cookies / Sessions
- Account Takeover
- Deface Websites
- Phishing & Redirection Data Theft

PREVENTION

Validate & Sanitize User Input
Always validate and sanitize all user input.

Output Encoding (Escape Data)
Encode special characters before rendering output.

Content Security Policy (CSP)
Use CSP to restrict sources of scripts and other content.

HttpOnly & Secure Cookies
Use HttpOnly and Secure flags for cookies.

Keep Frameworks & Libraries Updated
Regularly update to fix security vulnerabilities.

CROSS-SITE SCRIPTING (XSS) (2 OF 2)

The real-world impact of a successful XSS attack, and the prevention techniques covered across this module.

IMPACT OF XSS

- **Session Hijacking —** steal cookies or tokens to take over user accounts.
- **Data Theft —** access sensitive information visible in the user's browser.
- **Malicious Actions —** perform actions on behalf of the user (change email, transfer funds, etc.).
- **Website Defacement —** alter page content and damage reputation.

COMMON PREVENTION TECHNIQUES

- **Input Validation —** validate and allow only expected input, client and server.
- **Output Encoding —** encode special characters (<, >, &, ') before rendering output.
- **Use Framework Protection —** React escapes values in JSX by default.
- **Content Security Policy (CSP) —** restrict sources of scripts to reduce impact.
- **HttpOnly Cookies —** store session cookies with HttpOnly to prevent JS access.

LAB 36 XSS PROOF

xss.test.tsx renders <img onerror=alert(1)> as a customer's fullName and asserts no <img> node exists -- only literal text.

KEY TAKEAWAY Never trust user input. Validate on the server, encode output in the browser, and use security headers and secure cookies to protect against XSS.

PREVENTING XSS ATTACKS (1 OF 2)

The best defense against XSS is to assume all user input is untrusted. Validate, encode, and sanitize data at the right places to stop malicious scripts from executing.

1. VALIDATE USER INPUT

Validate input on the client for a better user experience and on the server for strong security. Allow only expected data types; enforce length, format, and pattern; use allowlists, not denylists.

Example (validation)

```
const isValid = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
```

2. ENCODE OUTPUT (CONTEXT-AWARE)

Context	What to Encode
HTML Content	Encode: < > & " '
HTML Attribute	Encode: " & < >
JavaScript	Encode special JS characters.
URL	Use encodeURIComponent().

3. USE SAFE FRAMEWORKS & TEMPLATING

Modern frameworks escape data by default. React escapes values in JSX by default; avoid inserting raw HTML via innerHTML, dangerouslySetInnerHTML, document.write(), or eval().

KEY TAKEAWAY Preventing XSS requires a combination of input validation, output encoding, secure configuration, and safe coding practices. Defense in depth is key.

Preventing XSS Attacks

Follow these best practices to protect your web application from Cross-Site Scripting (XSS) attacks.

1. Validate & Sanitize User Input

Always validate and sanitize all user input on the server side. Allow only expected data.

2. Encode Output (Escape Data)

Encode output based on the context (HTML, attribute, JS, CSS, URL) before rendering.

3. Use Safe APIs & Frameworks

Use modern frameworks and APIs that automatically escape output by default.

4. Avoid Inline JavaScript

Avoid inline JavaScript and event handlers. Use external JS files and strict policies.

5. Use Content Security Policy (CSP)

Implement CSP headers to restrict loading of scripts and other resources.

6. Use HttpOnly & Secure Cookies

Set cookies with HttpOnly and Secure flags to reduce the risk of theft via XSS.

7. Filter Dangerous Content

Use allowlists and filters to block potentially dangerous input like <script>, on* events, etc.

8. Keep Libraries & Dependencies Updated

Regularly update libraries, frameworks, and dependencies to patch security vulnerabilities.

KEY TAKEAWAY

Validate input, encode output, use security headers, and follow secure coding practices to build robust defenses against XSS attacks.

PREVENTING XSS ATTACKS (2 OF 2)

Content Security Policy, secure cookies, and dependency hygiene round out a defense-in-depth strategy against XSS.

4. USE CONTENT SECURITY POLICY (CSP)

CSP restricts where scripts can be loaded and executed from, helping prevent XSS even if a payload gets injected.

Example CSP header

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' https://trusted.cdn.com;
  object-src 'none'; base-uri 'self';
```

5. USE HTTPONLY & SECURE COOKIES

Store session tokens in HttpOnly cookies to reduce the risk of XSS stealing them via JavaScript. HttpOnly (no JS access), Secure (HTTPS only), SameSite (helps prevent CSRF).

```
Set-Cookie: token=abc123; HttpOnly; Secure; SameSite=Strict; Path=/
```

6. KEEP DEPENDENCIES & LIBRARIES UPDATED

- Regularly update dependencies and monitor security advisories.
- Remove unused packages; use tools like npm audit or Snyk.

LAB 36 EVIDENCE

Lab 36 Step 11 requires curl -I evidence showing Content-Security-Policy, X-Content-Type-Options: nosniff, and Referrer-Policy: no-referrer configured at the server or gateway.

KEY TAKEAWAY Preventing XSS requires a combination of input validation, output encoding, secure configuration, and safe coding practices. Defense in depth is the key to protecting your users and your application.

TRY IT YOURSELF — EXERCISE 3: XSS AND CSP NOTES

Reinforces: documenting XSS-safe rendering rules and a CSP note for customer names and notes -- a documentation exercise.

THE DANGER

- **Stored Payload —** if a malicious customer name contains <script>...</script> and the CRM UI renders it with dangerouslySetInnerHTML, the injected script can execute and steal the in-memory access token.

Paper test string (must render as literal text)

```
Amina <b>Khan</b>
```

STEPS (IN notes/lab36-security.md)

Step	What To Do
1 — Danger	Write one sentence: a malicious name with <script> rendered via dangerouslySetInnerHTML can steal tokens.
2 — Rule	Write the rule: prefer plain text children / React's default JSX escaping; avoid HTML-injection APIs.
3 — CSP	Write one sentence: CSP can reduce inline-script risk (the lab may only document this, not enforce it).
4 — Test Idea	Record the paper test string Amina Khan -- it must display as literal text, not bold.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-xss-csp.md (workspace: examples/module-36-exercises). Pass criteria: the dangerouslySetInnerHTML warning, the prefer-escaping rule, and the test string recorded.

CROSS-SITE REQUEST FORGERY (CSRF) (1 OF 2)

CSRF tricks a logged-in user's browser into performing unwanted actions on a web application where the user is authenticated. The attacker does not need to steal any data.

KEY POINTS

- **Targets authenticated users —** the attacker exploits the trust the site has in the user's browser.
- **No theft of data required —** the goal is to perform actions on behalf of the user.
- **Relies on cookies —** the browser automatically includes cookies in requests to the site.
- **Difficult to detect —** requests look legitimate to the server.

REAL-WORLD EXAMPLE

The attacker exploits the user's authenticated session to perform actions without the user's consent -- e.g. a malicious <img src="site.com/transfer?amount=1000"> tag, or an auto-submitted hidden form that fires the moment the victim's browser loads the malicious page.

HOW CSRF ATTACK WORKS

KEY TAKEAWAY CSRF exploits trust -- specifically, the trust a server places in cookies the browser sends automatically. This is why a Bearer-token API behaves differently from a cookie-session app.

Cross-Site Request Forgery (CSRF)

An attacker tricks a user's browser into performing unwanted actions on a trusted website where the user is authenticated.

1 USER LOGS IN

The user logs in to the trusted website.

2 SESSION CREATED

The website creates a session and stores a session cookie in the user's browser.

3 USER VISITS MALICIOUS SITE

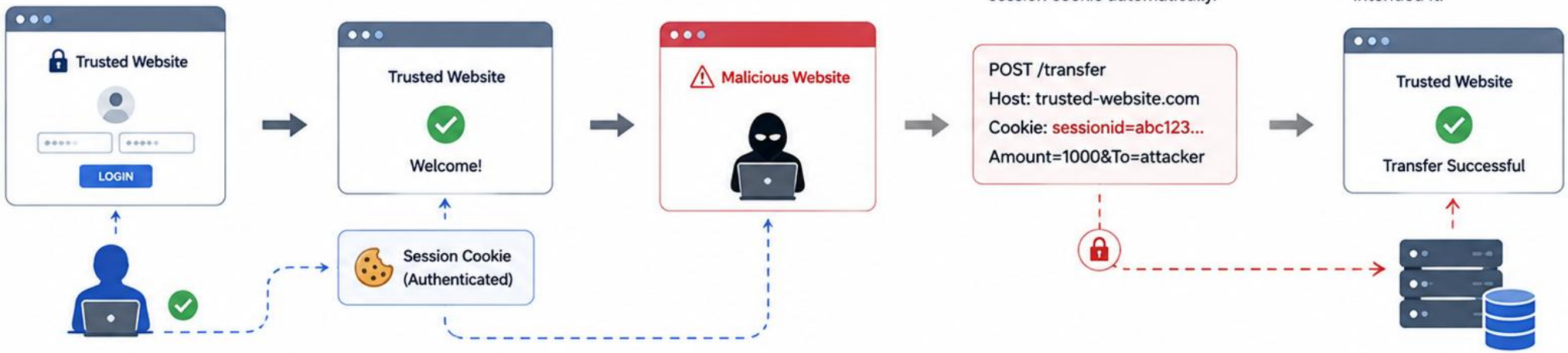
The user visits a malicious website controlled by the attacker.

4 HIDDEN REQUEST SENT

The malicious site sends a hidden request to the trusted website. The browser includes the session cookie automatically.

5 REQUEST PROCESSED

The trusted website receives the request and performs the action thinking the user intended it.



IMPACT

- Unauthorized actions on behalf of the user
- Data modification or deletion
- Account takeover
- Financial loss and security breaches

PREVENTION

Use Anti-CSRF Tokens
Include unique tokens in forms and validate them on the server.

SameSite Cookies
Set SameSite=Strict or SameSite=Lax for cookies.

Verify Origin/Referer
Validate the Origin or Referer header on state-changing requests.

Use POST for State-Changing Actions
Avoid GET requests for actions that modify data.

CROSS-SITE REQUEST FORGERY (CSRF) (2 OF 2)

The real-world impact of CSRF, common attack vectors, and how CSRF differs from XSS.

IMPACT OF CSRF

- **Financial Loss —** unauthorized transfers or payments.
- **Account Takeover —** change email, password, or security settings.
- **Data Manipulation —** modify or delete user data.
- **Reputation Damage —** loss of user trust and business impact.

HOW CSRF DIFFERS FROM XSS

Feature	CSRF	XSS
Goal	Act on behalf of the user	Steal data or act in the browser
User interaction?	No (often a link/image)	Usually yes
What's exploited?	Trust between browser & server	Lack of input/output validation

COMMON CSRF VECTORS

Malicious <img> tag

Auto-submitted hidden form

Malicious cross-site JavaScript

LAB 36'S BEARER-ONLY DESIGN

Because the CRM SPA attaches Authorization: Bearer explicitly (never relying on cookies), a malicious page cannot forge that header. Lab 36 documents this as N/A for CSRF, with the rationale written out -- not "CSRF does not exist."

KEY TAKEAWAY CSRF attacks exploit trust in automatically-sent cookies. A bearer-token API sidesteps classic CSRF because the malicious page cannot forge an explicit Authorization header -- but this reasoning must be written down, not assumed.

PREVENTING CSRF ATTACKS (1 OF 2)

The best defense against CSRF is to ensure that browsers can only perform actions that the user intentionally initiated. Combine multiple techniques for strong protection.

1. USE CSRF TOKENS

Include a unique, unpredictable token in every state-changing request: generate per session, include in forms/requests, validate on the server, regenerate after login.

Example (hidden field in form)

```
<input type="hidden" name="csrf_token" value="aZ8fY2...9kLmT" />
```

2. USE SAMESITE COOKIES

Set SameSite=Strict or Lax to restrict cookies from being sent with cross-site requests.

```
Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Strict
```

3. VERIFY ORIGIN / REFERER

Check the Origin or Referer header for state-changing requests. Accept requests only from your domain; reject requests with missing or untrusted origins.

LAB 36's ORIGIN CHECK (RELATED PATTERN)

```
const url = new URL(
  path.startsWith("http") ? path : `${API_URL}${path}`
);
if (url.origin === apiOrigin && token) {
  headers.set("Authorization", `Bearer ${token}`);
}
```

KEY TAKEAWAY Use multiple layers -- tokens, cookies, headers, and smart design -- to ensure that only legitimate, user-initiated requests can change data.

Preventing CSRF Attacks

Implement these best practices to protect your application from Cross-Site Request Forgery (CSRF) attacks.

1 Use Anti-CSRF Tokens

Generate a unique, unpredictable token for each session and include it in forms and state-changing requests.

2 Validate Tokens on Server

Verify that the token is valid, matches the user's session, and has not expired before processing the request.

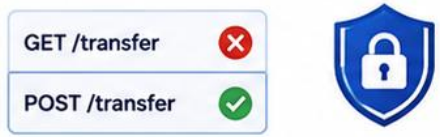
3 Use SameSite Cookies

Set SameSite=Lax or SameSite=Strict on cookies to prevent them from being sent with cross-site requests.

4 Verify Origin / Referer

Check the Origin or Referer header to ensure the request is coming from your own site.

5 Use POST for State-Changing Actions



Use POST (or PUT/PATCH/DELETE) for state-changing operations.
Avoid using GET for such actions.

6 Avoid GET Requests for Sensitive Actions

Do not perform sensitive or state-changing actions via GET requests.
They can be triggered by links or images.

7 Use Double-Submit Cookie Pattern

Send the CSRF token in a cookie and in a custom header. Validate that both tokens match on the server.

8 Expire Tokens and Rotate Regularly

Expire tokens after a short time and issue new tokens regularly to reduce risks.

Key Takeaway

Always verify the legitimacy of requests, use anti-CSRF tokens, and validate the source to protect users and your application.

PREVENTING CSRF ATTACKS (2 OF 2)

POST for state changes, the double submit cookie pattern, re-authentication for sensitive actions, and defense in depth.

4. USE POST FOR STATE-CHANGING ACTIONS

- Use POST (or PUT/DELETE) for actions that change data -- never GET.
- GET requests can be triggered easily (e.g. an <img> tag); POST is harder to forge.

5. DOUBLE SUBMIT COOKIE PATTERN

Send the CSRF token in both a cookie and a request header; the server compares both values.

```
Cookie: XSRF-TOKEN=abc123
Header: X-XSRF-TOKEN: abc123
```

6. RE-AUTHENTICATE FOR SENSITIVE ACTIONS

Require password or step-up auth for changing email/password, large transfers, or account deletion.

DEFENSE IN DEPTH

CSRF Tokens **SameSite Cookies** **Origin Check**

POST for Actions **Double Submit** **Re-authenticate**

LAB 36's COOKIE-MODE CSRF (STEP 10)

```
fetch(`${API_URL}/customers`, {
  method: "POST", credentials: "include",
  headers: {
    "Content-Type": "application/json",
    "X-XSRF-TOKEN": csrfToken,
  },
  body: JSON.stringify(draft),
});
```

KEY TAKEAWAY CSRF attacks exploit trust. Make sure every state-changing request is intentional, verifiable, and comes from your application -- and document which mode (bearer-only or cookie) your app actually uses.

TRY IT YOURSELF — EXERCISE 4: CSRF NOTES

Reinforces: matching CSRF applicability to the CRM SPA's actual token model -- a documentation exercise.

COOKIE SESSIONS vs BEARER HEADER

Model	CSRF Relevance
Cookie Session	If the auth cookie is sent automatically by the browser, CSRF is in scope -- needs SameSite and/or CSRF tokens.
Bearer Header (this lab)	If the token lives only in an explicit Authorization header set by JS, classic CSRF is reduced.

STEPS (IN notes/lab36-security.md)

Step	What To Do
1 — Cookie Sessions	Write: if the auth cookie is sent automatically, CSRF is in scope.
2 — Bearer Header	Write: if the token is only in an explicit Authorization header, classic CSRF is reduced.
3 — Lab Stance	State which model your Lab 36 starter follows, from the README or your instructor.
4 — Checklist Item	Add: apply SameSite cookie attributes if the app ever uses cookies.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-csrf-notes.md (workspace: examples/module-36-exercises). Pass criteria: cookie-vs-bearer contrast, a stated lab stance, and the SameSite checklist item.

PROTECTING SENSITIVE DATA (1 OF 2)

Sensitive data includes information that, if exposed, can harm users or the business. Protect it in transit, at rest, in use, and in client-side storage.

1. PROTECT DATA IN TRANSIT

- Use HTTPS everywhere; TLS 1.2 or higher.
- Enable HSTS to prevent downgrade attacks.
- Avoid sending sensitive data in URLs or query parameters.

2. PROTECT DATA AT REST

- Encrypt sensitive data stored in databases, files, and backups.
- Manage encryption keys securely (avoid hardcoding).

3. PROTECT DATA IN USE

- Minimize the amount of sensitive data handled on the client.
- Avoid storing sensitive data in memory longer than necessary.
- Mask sensitive data in the UI (e.g., passwords, card numbers).

4. SECURE CLIENT-SIDE STORAGE

- Avoid storing sensitive data in localStorage or sessionStorage.
- Never store passwords, PINs, or card details on the client.

KEY TAKEAWAY Data breaches lead to financial loss, legal consequences, reputational damage, and loss of user trust. Protect sensitive data at every stage.

Protecting Sensitive Data

Follow these best practices to keep sensitive data confidential, secure, and protected.

1 Encrypt Data in Transit

Use HTTPS/TLS to encrypt data when it travels between clients, servers, and third parties.

2 Encrypt Data at Rest

Encrypt sensitive data stored in databases, files, backups, and devices using strong algorithms (AES-256).

3 Minimize Data Exposure

Collect and store only the data you need. Mask, tokenize, or anonymize data when possible.

4 Access Control & Least Privilege

Restrict access to sensitive data based on roles and permissions. Grant the minimum access necessary.

5 Strong Authentication & Authorization

Use MFA and strong passwords. Validate users and authorize actions before granting access to sensitive data.

6 Use Secure Key Management

Securely generate, store, rotate, and revoke encryption keys. Never hardcode keys in code or configurations.

7 Secure Data Transmission & APIs

Validate and authenticate API requests. Use HTTPS, avoid exposing sensitive data in URLs, logs, or error messages.

8 Data Backup & Recovery

Encrypt backups and store them securely. Test recovery regularly to ensure data is available when needed.

9 Monitoring & Logging

Monitor access to sensitive data and log security events. Detect and respond to suspicious activities quickly.

10 Keep Systems & Software Updated

Regularly update systems, libraries, and dependencies to patch vulnerabilities and reduce risks.

KEY TAKEAWAY Protect sensitive data by encrypting it, controlling access, minimizing exposure, and continuously monitoring and updating your security practices.

PROTECTING SENSITIVE DATA (2 OF 2)

Secure communication and API design, classifying data by sensitivity, and monitoring for incidents.

5. SECURE COMMUNICATION & API

- Validate and sanitize all inputs; use strong authentication and authorization.
- Return only necessary data; use rate limiting to prevent abuse.
- Avoid verbose error messages that leak information.

Bad (Info Leakage)	Good (Minimal Info)
{"error":"User not found","email":"..."}	{"error":"Invalid credentials"}

6. DATA CLASSIFICATION

Classification	Examples
Public	Blog posts, help docs
Internal	Employee directory
Confidential	Financial reports, PII
Restricted	Credit card info, secrets

7. MONITOR & RESPOND

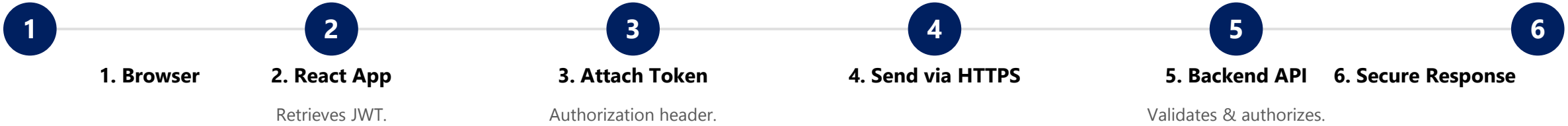
- Monitor access to sensitive data; log security events securely.
- Detect and alert on anomalies; have an incident response plan.

KEY TAKEAWAY Protect sensitive data at every stage -- transit, at rest, in use, and on the client. Use encryption, secure storage, minimal exposure, and strong access controls. Apply the principle of least privilege.

SECURING API CALLS (1 OF 2)

Securing API calls ensures that only authorized clients can access backend services and that data is protected in transit and at rest.

SECURE API CALL FLOW



KEY SECURITY MEASURES

- **Use HTTPS Everywhere —** encrypt data in transit and prevent eavesdropping or man-in-the-middle attacks.
- **Authenticate Every Request —** include a valid JWT in the Authorization header with every API call.
- **Validate Tokens on the Server —** verify signature, expiration, issuer, and audience.
- **Enforce Authorization (RBAC) —** check user roles and scopes before allowing access to resources.

KEY TAKEAWAY Secure API calls protect your application, your users, and your data. Security is a shared responsibility between frontend and backend.

Securing API Calls

Follow these best practices to protect your APIs and the data they exchange.

1 Use HTTPS Everywhere

Always use HTTPS/TLS to encrypt data in transit and prevent eavesdropping and man-in-the-middle attacks.

2 Authenticate Requests

Require strong authentication using tokens (JWT, OAuth 2.0, API keys) to verify the identity of clients.

3 Authorize Access

Enforce proper authorization and role-based access control (RBAC) to ensure users can access only what they are allowed to.

4 Validate Input Data

Validate and sanitize all input data to prevent injection attacks and other security vulnerabilities.

5 Use Secure Headers

Set security headers like HSTS, Content-Security-Policy, X-Content-Type-Options, and X-Frame-Options.

6 Rate Limiting & Throttling

Implement rate limiting and throttling to prevent abuse, brute force attacks, and Denial of Service (DoS).

7 Protect Sensitive Data

Do not expose sensitive data in API responses. Mask or omit confidential information.

8 Use Strong Token Mechanisms

Use short-lived tokens, refresh tokens, and secure storage. Invalidate tokens on logout or suspicion.

9 Log & Monitor API Activity

Log API requests and monitor for suspicious activity. Set up alerts for anomalies and security events.

10 Version & Maintain APIs Securely

Version your APIs and regularly update and patch dependencies to fix vulnerabilities.

KEY TAKEAWAY | Secure API calls by encrypting data, authenticating and authorizing users, validating inputs, and continuously monitoring for threats.

SECURING API CALLS (2 OF 2)

Common threats to API calls and how to mitigate them, plus the frontend implementation checklist Lab 36 grades against.

Threat	Risk	Mitigation
XSS	Token theft via injected scripts.	HttpOnly/memory storage, sanitize inputs, CSP.
CSRF	Unauthorized API calls on behalf of the user.	SameSite cookies, CSRF tokens, origin headers.
Man-in-the-Middle (MITM)	Interception of requests and responses.	Enforce HTTPS and HSTS.
Token Replay	Reuse of stolen tokens.	Short-lived access tokens with rotation.
Brute Force / Guessing	Unauthorized access via repeated attempts.	Rate limiting, account lockout, strong auth.

QUICK CHECKLIST

- Use HTTPS for all API calls.
- Attach JWT in Authorization header (origin-scoped).
- Validate token on every server-side request.
- Protect against XSS, CSRF, and MITM.
- Store tokens securely (memory, never localStorage).
- Handle 401 vs 403 differently; log out cleanly.

KEY TAKEAWAY Secure API calls protect your application, your users, and your data. Security is a shared responsibility -- the frontend attaches credentials correctly; the backend validates everything.

TRY IT YOURSELF — EXERCISE 5: FILL ROUTE GUARD TODOS (STARTER)

Reinforces: completing a RequireAuth wrapper's TODOs and reasoning about what a route guard can and cannot enforce -- a hands-on starter exercise.

STARTER CODE — FILL EVERY BLANK

notes/lab36-todos.md (starter -- not a complete solution)

```
function RequireAuth({ children }: { children: React.ReactNode }) {
  const token = _____; // read from your chosen storage
  if (!token) {
    return <Navigate to="_____" replace />;
  }
  return _____;
}

// TODO: attach Authorization: Bearer _____ on API fetch (Lab 35+36)
// TODO: never log full token; log correlation lab-request-001 instead
```

STEPS (IN notes/lab36-todos.md)

Step	What To Do
1 — Paste	Create notes/lab36-todos.md and paste the starter RequireAuth stub above.
2 — Fill Blanks	Suggested fills: getAccessToken(), "/login", <>{children}</>, and the fake token lab-token-001.
3 — Role Note	Add the optional TODO: hide AdminMenu unless role===ADMIN -- label it UI only.
4 — Backend Reminder	Write: Spring Security must still reject unauthorized API calls for CUS data, even if the guard passes.

KEY TAKEAWAY TRY IT NOW (STARTER) — Complete Exercise 5 before continuing: exercises/exercise-05-fill-guard-todos.md (workspace: examples/module-36-exercises). Every _____ and // TODO must be replaced -- this is a fill-in-the-blank starter, not a finished solution.

AUTHENTICATION FLOWS (1 OF 2)

Authentication flows define how users prove their identity and receive access to your application and APIs. Choose the right flow for your app type and needs.

1. SESSION-BASED AUTHENTICATION

- Login request validated against the database; server creates a session and sets a session cookie.
- Subsequent requests carry the session cookie automatically.

Best for: server-rendered applications with same-origin requests. State: stateful. CSRF risk: high (mitigate with CSRF tokens).

2. TOKEN-BASED AUTHENTICATION (JWT)

- Login request validated; auth server issues a JWT; client stores it (memory or secure storage).
- Client attaches the JWT to every API request; server validates it statelessly.

Best for: SPAs, mobile apps, and APIs -- stateless, scalable, and secure. This is the flow the CRM SPA and Lab 36 use.

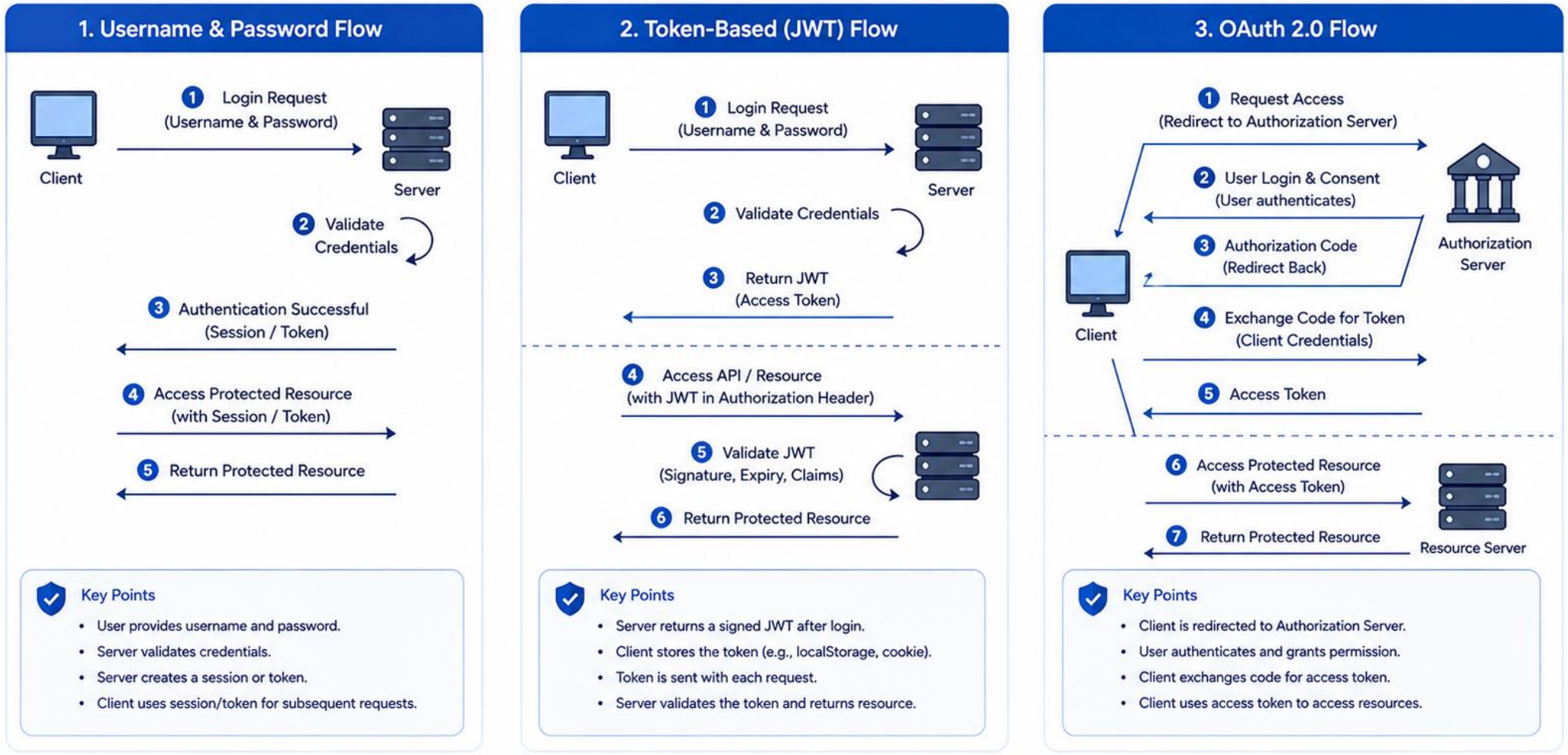
COMMON BEST PRACTICES ACROSS ALL FLOWS

Always Use HTTPS | **Store Tokens Securely** | **Expiration & Refresh** | **Strong Passwords & MFA** | **Log Auth Events**

KEY TAKEAWAY Choose the right authentication flow based on your application architecture and security requirements. The CRM SPA uses token-based (JWT) authentication -- stateless, scalable, and a natural fit for a React frontend calling a Spring API.

Authentication Flows

Common authentication flows used in modern web applications



AUTHENTICATION FLOWS (2 OF 2)

OAuth 2.0 for third-party login, refresh token flow for long sessions, and a side-by-side comparison of all four flows.

3. OAUTH 2.0 AUTHORIZATION CODE FLOW

- App redirects to a third-party auth server (Google, GitHub, etc.); user authenticates there.
- Auth server returns an authorization code, exchanged for an access token.

Best for: access to third-party services. State: stateless. CSRF risk: low.

4. REFRESH TOKEN FLOW

- Client receives an access token + refresh token at login.
- When the access token expires, the refresh token is exchanged for a new one -- no re-login needed.

Best for: maintaining sessions securely without forcing re-login. CSRF risk: low.

FLOW COMPARISON

Flow	State	Scalability	Best Use Case	Token Used	CSRF Risk
Session-Based	Stateful	Moderate	Traditional Web Apps	Session ID (Cookie)	High
Token-Based (JWT)	Stateless	High	SPAs, Mobile Apps, APIs	JWT	Low
OAuth 2.0 (Auth Code)	Stateless	High	Third-Party Integrations	Access Token	Low
Refresh Token Flow	Stateless	High	Long-Lived Sessions	Access + Refresh	Low

KEY TAKEAWAY Strong authentication is the first line of defense. Secure the flow, protect the tokens, and monitor continuously -- the CRM SPA's own flow is Token-Based (JWT), row 2 of this table.

FRONTEND SECURITY BEST PRACTICES

The frontend is the first line of interaction with users. Follow these best practices consistently to protect your application and your users.

TEN PRACTICES TO APPLY CONSISTENTLY

1. Validate & Sanitize Input

Validate on client and server; sanitize before displaying; use allowlists.

2. Prevent XSS

Never trust user input; encode output; avoid innerHTML; use CSP.

3. Prevent CSRF

Anti-CSRF tokens for state changes; SameSite cookies; validate Origin.

4. Secure Auth & Sessions

HttpOnly, SameSite cookies or memory tokens; short expiration; clean logout.

5. Protect Sensitive Data

Never expose secrets in the frontend; mask sensitive fields; use HTTPS.

6. Secure Dependencies

Keep dependencies updated; scan for vulnerabilities; remove unused packages.

7. Implement CSP

Restrict script/style/image/iframe sources; prevent inline scripts and eval().

8. Secure API Communication

HTTPS for all calls; validate certificates; handle errors gracefully.

9. Avoid Insecure Storage

Avoid localStorage/sessionStorage; use HttpOnly cookies instead.

10. Monitor & Log Errors

Log securely; friendly errors to users; monitor suspicious activity.

KEY TAKEAWAY A secure frontend is built by design, not by chance. Golden rules: Validate Everything, Trust Nothing, Encode Output, Secure Secrets, Stay Updated & Monitor Continuously.

Frontend Security Best Practices

Secure your frontend applications by following these best practices.

1 Use HTTPS Everywhere

- Always use HTTPS to prevent eavesdropping and man-in-the-middle attacks.

2 Validate and Sanitize User Input

- Validate input on both client and server.
- Sanitize data to prevent injection attacks (XSS).

3 Prevent XSS Attacks

- Encode output data.
- Avoid innerHTML with untrusted content.
- Use Content Security Policy (CSP).

4 Prevent CSRF Attacks

- Use Anti-CSRF tokens in forms and requests.
- Set SameSite cookies to Strict or Lax.

5 Secure Authentication and Session

- Use strong authentication mechanisms (OAuth2, JWT).
- Store tokens securely and avoid exposing them.

6 Secure Token Storage

- Store tokens in HttpOnly, Secure, SameSite cookies.
- Avoid storing tokens in localStorage or sessionStorage.

7 Protect Sensitive Data

- Do not expose sensitive data in the frontend.
- Mask or encrypt sensitive information.

8 Use Secure Headers

- ✓ Content-Security-Policy
- ✓ Strict-Transport-Security
- ✓ X-Content-Type-Options
- ✓ Referrer-Policy

- Enforce security headers via server configuration.
- Helps mitigate common web vulnerabilities.

9 Keep Dependencies Up to Date

- Regularly update libraries and frameworks.
- Fix known vulnerabilities promptly.

10 Follow Principle of Least Privilege

- Request only the necessary permissions and data.
- Limit access and functionality based on user roles.

Key Takeaway

Secure by default, validate always, and trust never.
Implement these practices to build robust and secure frontend applications.

SECURE DEVELOPMENT CHECKLIST

Use this checklist throughout the frontend development lifecycle to build secure, resilient, and trustworthy web applications.

TEN CHECKPOINTS ACROSS THE DEVELOPMENT LIFECYCLE

1. Input Validation

Validate client & server; sanitize and encode output. Never trust user input.

2. Authn & Authz

Strong auth, MFA, RBAC. Verify identity; enforce least privilege.

3. Secure API Comm.

HTTPS, validate SSL, Bearer headers, safe error handling.

4. Session & Token

HttpOnly/Secure/SameSite when possible; short expiration; clear on logout.

5. Prevent Web Attacks

Sanitize/encode for XSS; anti-CSRF + SameSite; CSP for clickjacking.

6. Protect Sensitive Data

Never hardcode secrets/keys/tokens; mask PII; encrypt in transit.

7. Secure Dependencies

Keep updated; scan regularly; remove unused packages/code.

8. Content Security Policy

Restrict resource loading; avoid inline scripts/unsafe-eval.

9. Error Handling & Logging

Handle errors gracefully; no stack traces to users; log events.

10. Testing & Monitoring

Regular security testing; monitor and review continuously.

KEY TAKEAWAY Security is a shared responsibility -- you build it, you run it, you own it. Attackers only need one mistake; your users trust you to protect them. Build security in from the start, verify continuously.

Secure Development Checklist

Follow this checklist to build secure, reliable, and resilient applications.

1 Authentication & Authorization

- Implement strong authentication (MFA, strong passwords).
- Use proven mechanisms (OAuth2, JWT).
- Enforce role-based access control (RBAC).

2 Input Validation

- Validate and sanitize all user inputs.
- Use allowlists over denylists.
- Prevent injection attacks (XSS, SQLi, NoSQLi, etc.).

3 Secure Coding Practices

- Follow secure coding standards and guidelines.
- Avoid hardcoding secrets and sensitive data.
- Handle errors securely and avoid exposing details.

4 Prevent Common Vulnerabilities

- Protect against XSS, CSRF, SSRF, IDOR, and Clickjacking.
- Use prepared statements and parameterized queries.
- Encode output and set security headers.

5 Data Protection

- Encrypt sensitive data in transit (HTTPS/TLS).
- Encrypt sensitive data at rest.
- Minimize collection and exposure of personal data.

6 Session & Token Security

- Use secure, HttpOnly, SameSite cookies.
- Set appropriate token expiration and rotation.
- Invalidate sessions/tokens on logout.

7 Dependencies & Components

- Use trusted sources for libraries and packages.
- Keep dependencies up to date.
- Scan for known vulnerabilities regularly.

8 Testing & Quality Assurance

- Perform SAST, DAST, and dependency scanning.
- Write unit and integration tests.
- Conduct code reviews and security testing.

9 Logging & Monitoring

- Log security events and errors (without sensitive data).
- Monitor for suspicious activity and anomalies.
- Set up alerts and incident response.

10 Deployment & Operations

- Use secure configurations and infrastructure.
- Enable security headers and HTTPS.
- Regularly backup data and test recovery.

Key Takeaway | Security is a continuous process. Build secure, test regularly, and stay updated to protect your users and data.

TRY IT YOURSELF — EXERCISE 6: LAB 36 READINESS

Reinforces: confirming your security decisions are written down before you touch guard code -- a pre-lab capstone self-check.

DECISION LOG — WRITE THESE THREE BEFORE CODING

- **Storage Choice —** which token storage you picked in Exercise 2, and why.
- **Guard Redirect Target —** where RequireAuth sends an unauthenticated user, from Exercise 5.
- **XSS Rule —** the prefer-escaping rule you wrote in Exercise 3.

STEPS (SELF-CHECK)

Step	What To Do
1 — Decision Log	List your three decisions: storage choice, guard redirect target, and XSS rule.
2 — Evidence Preview	Preview the future evidence: an unauthenticated visit redirects to /login; the XSS test string renders safely.
3 — No Real IdP	Confirm: this pre-lab does not require configuring a real IdP such as Okta or Auth0.
4 — Pass Mark	Mark yourself Pass only if your Exercise 5 guard TODOs are fully filled in.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 6 before Lab 36: exercises/exercise-06-lab36-readiness.md (workspace: examples/module-36-exercises). Pass criteria: three decisions listed, IdP setup deferred, and a Pass/Fail self-mark.

LAB OVERVIEW -- FRONTEND SECURITY FOR THE CRM SPA

Lab 36: Frontend Security for the CRM SPA -- threat model, memory tokens, origin-scoped auth, XSS proof, CSRF/CSP evidence.

BUSINESS SCENARIO

- Customer PII in the CRM SPA is a high-value browser asset. Attackers aim for token theft, XSS, CSRF (cookie mode), and open redirects after login.
- Leadership freezes: no merge of CRM SPA auth without a written threat model, in-memory token discipline, origin-scoped Authorization, XSS-safe rendering tests, and clear 401 vs 403 behavior.
- Spring must authorize every API call; React route guards only improve UX -- they are never the security boundary.

LEARNING OBJECTIVES

- Model JWT/session, XSS, CSRF, and browser threats for the CRM SPA.
- Create explicit auth state (checking / anonymous / authenticated); store tokens in memory only.
- Attach bearer tokens only to the approved CRM API origin.
- Protect navigation while treating guards as UX -- not authorization.
- Handle 401 (expire), 403 (forbidden), and logout completely; render data without HTML sinks.

WHAT YOU BUILD

- lab36-crm/crm-ui: docs/security-decisions.md threat model; AuthContext + in-memory tokenStore; origin-scoped Authorization in http.request; LoginPage + ProtectedRoute; xss.test.tsx and security.test.tsx; CSP/security-header evidence.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; correlation ID lab-request-001 appears on authenticated calls. Backend authorization remains the source of truth throughout.

LAB TASKS AND DELIVERABLES

Frontend Security for the CRM SPA -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Write the threat model (docs/security-decisions.md): assets, boundaries, attacker goals, controls.
2	Model authentication state: checking / anonymous / authenticated (AuthContext).
3	Create an in-memory token store (tokenStore.ts) -- never localStorage/sessionStorage.
4	Restrict bearer-token destinations: attach Authorization only when url.origin === apiOrigin.
5	Build a safe login form: generic errors; reject external returnUrl / open redirects.
6	Guard client navigation with ProtectedRoute (UX only -- API still returns 401 without a token).
7	Distinguish 401 (clear token, expire) from 403 (keep session, forbidden UX).
8	Implement complete logout: revoke, clear token + customer cache, navigate to /login.
9	Prove customer text cannot execute: xss.test.tsx against a malicious fullName.
10	Add cookie-mode CSRF protection, or document N/A with rationale for bearer-only.
11	Add browser security headers (CSP, nosniff, referrer-policy); capture curl -I evidence.
12	Run security.test.tsx abuse tests + green build twice; redacted screenshots.

EXPECTED DELIVERABLES

- Threat model document.
- Auth state + in-memory token store.
- Origin-scoped Authorization on CRM calls.
- Login + ProtectedRoute + 401/403/logout behavior.
- XSS proof test; CSRF evidence or N/A rationale.
- CSP/security headers evidence.
- Abuse tests + green build; redacted screenshots + README runbook.

RUBRIC (100 MARKS)

- Env/Structure 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Automated Verification 10 · Security/Prod Awareness 10 · Docs/Evidence 10

KEY TAKEAWAY localStorage tokens left in submission are an honor violation. Claiming ProtectedRoute authorizes APIs fails the security marks.

MODULE 36 SUMMARY

Frontend security is critical to protecting users, applications, and sensitive data. This module covered key concepts, best practices, and a hands-on lab hardening the CRM SPA.

KEY CONCEPTS COVERED

Threats & OWASP Risks

The frontend attack surface and the OWASP risks most relevant to client-side code.

JWT & Token Storage

How JWT authentication works, and safely storing, expiring, and refreshing tokens.

XSS & CSRF Defense

How each attack works and the layered defenses that prevent them.

Secure API Calls

Authenticating every request and enforcing authorization on the backend.

Authentication Flows

Session, token, OAuth 2.0, and refresh-token patterns and their trade-offs.

Best Practices & Checklist

A practical, ongoing checklist for building security in by design.

MODULE FLOW RECAP

1. **Threats**
2. **JWT & Tokens**
3. **XSS & CSRF**
4. **API Calls**
5. **Lab: CRM SPA**

KEY TAKEAWAY A secure frontend protects users and data by preventing attacks, validating inputs, handling authentication securely, and following best practices throughout the development lifecycle. Never trust the client -- the server authorizes every call.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of frontend security concepts and best practices: XSS goals, CSRF prevention, HTTPS, and secure token storage.

1. Which is the primary goal of preventing XSS attacks?

A. Prevent unauthorized access to servers

B. Avoid stealing user sessions or injecting malicious scripts

C. Encrypt data in transit

D. Improve website performance

2. Which technique is most effective in preventing CSRF attacks?

A. Disabling cookies

B. Using anti-CSRF tokens

C. Using local storage

D. Hiding buttons in the UI

3. HTTPS provides which of the following security benefits?

A. Protects data confidentiality and integrity in transit

B. Prevents SQL injection

C. Protects against brute force login

D. Ensures data is stored in the browser

4. Why should sensitive tokens be stored in HttpOnly cookies instead of localStorage?

A. HttpOnly cookies are encrypted

B. Inaccessible to JavaScript, reducing XSS risk

C. localStorage is slower than cookies

D. HttpOnly cookies increase page load speed

KEY TAKEAWAY Preventing XSS means stopping stolen sessions or injected scripts; anti-CSRF tokens defend against CSRF; HTTPS protects confidentiality and integrity in transit; HttpOnly cookies block JavaScript access, reducing XSS risk to the token.

KNOWLEDGE CHECK (2 OF 2)

Two more multiple-choice questions on clickjacking and API error handling, plus a True/False review of key module concepts.

5. Which HTTP header helps protect against clickjacking attacks?

A. Content-Security-Policy

B. X-Frame-Options

C. Strict-Transport-Security

D. Referrer-Policy

6. What is the best practice for handling errors in API calls on the frontend?

A. Display full error details to the user

B. Log errors securely and show user-friendly messages

C. Ignore errors to avoid exposing information

D. Redirect users to a blank page

TRUE OR FALSE

Statement	Answer
Content Security Policy (CSP) helps prevent XSS attacks by controlling which resources can be loaded.	TRUE
Storing JWT tokens in localStorage is the most secure way to store tokens.	FALSE
Rate limiting API calls can help reduce the impact of abuse.	TRUE
Frontend validation alone is sufficient to secure an application.	FALSE

KEY TAKEAWAY X-Frame-Options defends against clickjacking; log errors securely with user-friendly messages; CSP helps prevent XSS; localStorage is NOT the most secure token store; rate limiting helps; frontend validation alone is never sufficient.

"Never trust the client -- verify everything on the server."

MODULE 36 COMPLETE

Secure Frontend Communication

You can now identify frontend threats, handle JWTs and tokens securely, defend against XSS and CSRF, and secure API calls in a React single-page application.